Лабораторная работа №3

Базовые растровые алгоритмы

Отчёт

Студент: Герасимович Данила

Группа: 4, ФМиИС

Октябрь 2026

Contents

1 Введение **3**

1.1 Цель работы 3

1.2 Постановка задачи 3

1.3 Среда разработки 3

2 Теоретическая часть **4**

2.1 Понятие растеризации 4

2.2 Пошаговый алгоритм 4

2.3 Алгоритм ЦДА (DDA) 4

2.4 Алгоритм Брезенхема для отрезка 5

2.5 Алгоритм Брезенхема для окружности 5

3 Реализация **5**

3.1 Архитектура приложения 5

3.2 Привязка координат к дискретной сетке 6

3.3 Согласование правил округления 6

3.4 Пользовательский интерфейс 7

4 Тестирование и примеры вычислений **7**

4.1 Пошаговый алгоритм 7

4.2 Алгоритм ЦДА 8
4.3 Алгоритм Брезенхема для отрезка 8
4.4 Случай, когда алгоритмы расходятся 9
4.5 Алгоритм Брезенхема для окружности 9

5 Временные характеристики 10

6 Реализация дополнительных возможностей 11
6.1 Анимация 11
6.2 Масштабирование 11
6.3 Замер времени 11

7 Заключение 11

А Ключевые фрагменты кода 12
А.1 Алгоритм Брезенхема для отрезка 12
А.2 Алгоритм Брезенхема для окружности 13
А.3 Привязка координат к дискретной сетке 13
А.4 Округление, совместимое с эталоном 13
А.5 Выводы 14

1 Введение

1.1 Цель работы

Закрепление теоретического материала и практическое освоение базовых алгоритмов растеризации отрезков и кривых:

- пошаговый алгоритм;
- алгоритм ЦДА (цифровой дифференциальный анализатор);
- алгоритм Брезенхема для отрезка;
- алгоритм Брезенхема для окружности.

1.2 Постановка задачи

Требуется разработать настольное приложение, иллюстрирующее работу перечисленных алгоритмов, с удобным интерфейсом, сеткой, осями координат, масштабированием, анимацией построения и замером времени выполнения. Также необходимо привести один пример с сопроводительными вычислениями для каждого алгоритма, составить краткий отчёт о временных характеристиках и пояснить, каким образом целочисленные координаты привязаны к дискретной сетке.

1.3 Среда разработки

Приложение реализовано в виде единого файла `main.py`, содержащего весь код и запускающего графическое окно. Используется язык Python 3, стандартная библиотека Tkinter (модули `tkinter`, `tkinter.ttk`) и модуль `time` для замеров. Внешних зависимостей нет — приложение запускается в любой системе, где установлен Python 3.10 или новее с Tkinter в комплекте.

2 Теоретическая часть

2.1 Понятие растеризации

Растеризация (сканирующее преобразование) — процесс преобразования геометрического примитива, заданного в непрерывных координатах, в набор пикселей дискретного растра. Качество растеризации определяется близостью получающегося набора пикселей к идеальной математической линии и отсутствием визуально заметных разрывов.

2.2 Пошаговый алгоритм

Идея пошагового (прямого) алгоритма состоит в том, что вдоль ведущей оси (той, по которой приращение координаты больше) выполняется шаг ровно на 1 пиксель, а вторая координата вычисляется по уравнению прямой и округляется:

$$y = y_1 + \frac{dy}{dx}\,(x - x_1), \quad dx = x_2 - x_1,\ dy = y_2 - y_1.$$

Если $|dx| \geq |dy|$, ведущей является ось x; в противном случае — ось y. Главный недостаток метода — операция деления с плавающей точкой на каждом шаге.

2.3 Алгоритм ЦДА (DDA)

Цифровой дифференциальный анализатор основан на вычислении приращений:

$$\Delta x = \frac{x_2 - x_1}{\text{steps}}, \quad \Delta y = \frac{y_2 - y_1}{\text{steps}}, \quad \text{steps} = \max(|x_2 - x_1|,\ |y_2 - y_1|).$$

На каждом шаге координаты увеличиваются на эти приращения и округляются до целых:

$$x_{i+1} = x_i + \Delta x, \quad y_{i+1} = y_i + \Delta y.$$

Алгоритм также использует арифметику с плавающей точкой, но требует лишь $O(\max(|dx|, |dy|))$ шагов и не выполняет деления внутри цикла.

2.4 Алгоритм Брезенхема для отрезка

Алгоритм Брезенхема работает только с целочисленной арифметикой. Основная идея — на каждом шаге выбирать из двух соседних пикселей тот, который ближе к идеальной линии. Для отрезка с $|dx| \geq |dy|$ вводятся:

$$dx = |x_2 - x_1|, \quad dy = -|y_2 - y_1|, \quad err = dx + dy.$$

На каждом шаге:

$e_2 = 2\,err$; если $e_2 \geq dy$, то $err \mathrel{+}= dy,\ x \mathrel{+}= s_x$; если $e_2 \leq dx$, то $err \mathrel{+}= dx,\ y$

Здесь $s_x = \text{sign}(x_2 - x_1)$, $s_y = \text{sign}(y_2 - y_1)$. Алгоритм работает за $O(|dx| + |dy|)$ шагов и не использует операций с плавающей точкой.

2.5 Алгоритм Брезенхема для окружности

Для окружности радиуса R с центром (x_c, y_c) используется свойство симметрии: достаточно построить один октант (от $x = 0, y = R$ до $x = y = R/\sqrt{2}$), после чего остальные точки получаются отражениями:

$$(\pm x, \pm y), \quad (\pm y, \pm x).$$

Начальное значение решающей функции: $d = 3 - 2R$. На каждом шаге при $x \to x + 1$:

$$\text{если } d < 0,\ d \mathrel{+}= 4x + 6, \quad \text{иначе } d \mathrel{+}= 4(x - y) + 10,\ y \mathrel{-}= 1.$$

Так же, как и для отрезка, все вычисления целочисленные.

3 Реализация

3.1 Архитектура приложения

Приложение реализовано как настольное приложение на Python 3 с графическим интерфейсом Tkinter. Холст `tk.Canvas` размером 900×600 пикселей используется

для вывода растровых точек и системы координат; для пошаговой таблицы точек применён виджет `ttk.Treeview`. Все алгоритмы реализованы в виде отдельных функций, возвращающих список кортежей `(x, y)` в мировых координатах; отрисовка выполняется отдельным универсальным методом.

Структура модулей:

- `read_params()` — чтение параметров из полей ввода;
- `world_to_screen(x, y)` — преобразование мировых координат в экранные;
- `draw_grid()` — отрисовка сетки, осей и подписей;
- `step_line`, `dda_line`, `bresenham_line`, `bresenham_circle` — реализации четырёх алгоритмов;
- `draw()`, `animate()`, `timing()` — управляющие методы.

3.2 Привязка координат к дискретной сетке

Пусть масштаб равен s пикселей на единицу мировой координаты, холст имеет размер $W \times H$, начало координат находится в центре холста. Тогда мировая точка (x, y) переводится в экранную:

$$s_x = \frac{W}{2} + x\,s, \quad s_y = \frac{H}{2} - y\,s.$$

Знак минус перед y соответствует направлению оси Y вверх (в декартовой системе), тогда как на экране ось Y направлена вниз. Каждая целочисленная точка (x, y) изображается квадратом со стороной s и центром в (s_x, s_y), что визуально соответствует одной ячейке сетки. Значения x и y пробегают целочисленные узлы, поэтому координаты дискретны и однозначно привязаны к сетке.

3.3 Согласование правил округления

В Python встроенная функция `round()` использует банковское округление (округление к чётному), тогда как в эталонной реализации на JavaScript функция `Math.round()` округляет половину от нуля. Чтобы сохранить пиксельную идентичность результатов, введён вспомогательный метод

```
def js_round(v):
    return int(math.floor(v + 0.5))
```

Listing 1: js_round(v) — округление в стиле Math.round

который применяется во всех местах, где по смыслу требуется «школьное» округление: в пошаговом алгоритме, в ЦДА и при преобразовании координат. Без этого метода расхождения с эталоном проявлялись бы на граничных случаях вида $y = -0{,}5$.

3.4 Пользовательский интерфейс

Интерфейс включает:

- выпадающий список выбора алгоритма (`ttk.Combobox`);
- поля ввода x_1, y_1, x_2, y_2 (или центра и радиуса для окружности);
- регулятор масштаба 10–80 пикселей на единицу (`tk.Scale`);
- флажок отображения сетки;
- кнопки: *Построить*, *Анимация*, *Замер времени*, *Очистить*;
- информационную панель с количеством точек;
- таблицу растровых точек (`ttk.Treeview`) с подсветкой текущего шага при анимации.

4 Тестирование и примеры вычислений

Для демонстрации корректности все четыре алгоритма проверены на одинаковых входных данных: отрезок $(2, 3) \to (11, 7)$ и окружность с центром $(0, 0)$ и радиусом 5.

4.1 Пошаговый алгоритм

Дано: $x_1 = 2$, $y_1 = 3$, $x_2 = 11$, $y_2 = 7$, $dx = 9$, $dy = 4$. Так как $|dx| > |dy|$, шагаем по x. Значение y вычисляется по формуле $y = 3 + \frac{4}{9}(x - 2)$ и округляется:

x	2	3	4	5	6	7	8	9	10	11
$y_{\text{неокругл.}}$	3.00	3.44	3.89	4.33	4.78	5.22	5.67	6.11	6.56	7.00
$y_{\text{округл.}}$	3	3	4	4	5	5	6	6	7	7

Получаем точки:

$$(2,3),(3,3),(4,4),(5,4),(6,5),(7,5),(8,6),(9,6),(10,7),(11,7).$$

4.2 Алгоритм ЦДА

steps $=\max(9,4)=9$, $\Delta x=1$, $\Delta y=4/9\approx 0{,}4444$. Накопление координат:

i	0	1	2	3	4	5	6	7	8	9
x	2.00	3.00	4.00	5.00	6.00	7.00	8.00	9.00	10.00	11.00
y	3.000	3.444	3.889	4.333	4.778	5.222	5.667	6.111	6.556	7.000
round(y)	3	3	4	4	5	5	6	6	7	7

Результат совпадает с пошаговым алгоритмом:

$$(2,3),(3,3),(4,4),(5,4),(6,5),(7,5),(8,6),(9,6),(10,7),(11,7).$$

4.3 Алгоритм Брезенхема для отрезка

$dx=9$, $dy=-4$, $s_x=1$, $s_y=1$, $err=5$. Пошаговая таблица (начало):

№	x	y	err	e_2	Действие
1	2	3	5	10	$e_2\ge -4\Rightarrow x{+}{=}1$; $e_2\le 9$ — нет
2	3	3	1	2	$x{+}{=}1$; $e_2\le 9\Rightarrow y{+}{=}1$
3	4	4	6	12	$x{+}{=}1$; $e_2\le 9$ — нет
4	5	4	2	4	$x{+}{=}1$; $y{+}{=}1$
…					

Получаемые точки:

$$(2,3),(3,3),(4,4),(5,4),(6,5),(7,5),(8,6),(9,6),(10,7),(11,7).$$

Замечание. На данном конкретном примере все три алгоритма (пошаговый, ЦДА и Брезенхем) дают идентичную последовательность пикселей. Это типично для «общих» положений прямой: алгоритм Брезенхема по построению выбирает ближайший к идеальной линии пиксель, а пошаговый и ЦДА с правильным округлением приходят к тому же результату. Принципиальная разница между ними — не в форме растра, а в *стоимости*: Брезенхем не выполняет ни одного деления и ни одного округления с плавающей точкой.

4.4 Случай, когда алгоритмы расходятся

Чтобы увидеть реальное расхождение по пикселям, рассмотрим отрезок $(0,0) \to (2,-1)$.

Алгоритм	$x=0$	$x=1$	$x=2$
Пошаговый	$(0,0)$	$(1,0)$	$(2,-1)$
ЦДА	$(0,0)$	$(1,0)$	$(2,-1)$
Брезенхем	$(0,0)$	$(1,-1)$	$(2,-1)$

Причина расхождения — пиксель на средней колонке, где идеальное значение $y=-0{,}5$ попадает точно на границу между двумя пикселями. Пошаговый и ЦДА используют `js_round(-0,5)` $=0$ (округление «от нуля вверх»), а правило Брезенхема переводит ошибку в направлении, соответствующем текущему знаку err, и выбирает $y=-1$. Оба ответа «правильны» по своим критериям; расхождение — следствие разных правил разрешения ничьей.

4.5 Алгоритм Брезенхема для окружности

Центр $(0,0)$, $R=5$. Начальные значения: $x=0$, $y=5$, $d=3-2\cdot 5=-7$. Пошагово:

x	y	d	Действие
0	5	-7	$d<0 \Rightarrow d{+}{=}4\cdot 0+6=6$ (стало -1)
1	5	-1	$d<0 \Rightarrow d{+}{=}10$ (стало 9)
2	5	9	$d\geq 0 \Rightarrow d{+}{=}4(2-5)+10=-2$ (стало 7), $y{=}4$
3	4	7	$d\geq 0 \Rightarrow d{+}{=}4(3-4)+10=6$ (стало 13), $y{=}3$

После применения симметрии по восьми октантам и удаления дубликатов (через множество пар (x, y)) получаем 28 уникальных точек:

$$(0,5),(5,0),(0,-5),(-5,0),\ (1,5),(5,1),(5,-1),(1,-5),$$
$$(-1,-5),(-5,-1),(-5,1),(-1,5),\ (2,5),(5,2),(5,-2),(2,-5),$$
$$(-2,-5),(-5,-2),(-5,2),(-2,5),\ (3,4),(4,3),(4,-3),(3,-4),$$
$$(-3,-4),(-4,-3),(-4,3),(-3,4).$$

5 Временные характеристики

Замер времени осуществляется встроенной функцией `time.perf_counter()`, обеспечивающей наносекундную точность на монотонных часах. Для каждого алгоритма выполняется N повторных построений (для отрезков $N = 2000$, для окружности $N = 500$), после чего вычисляется среднее время одного построения. Результаты зависят от версии интерпретатора Python, процессора и загрузки системы; в таблице 1 приведены характерные значения для CPython 3.13 на машине с процессором Intel Core i5.

Table 1: Усреднённое время построения одного примитива (Python/Tkinter)

Алгоритм	Повторов	Среднее время, мс	Кол-во точек
Пошаговый (отрезок)	2000	$\sim 10 \cdot 10^{-3}$	10
ЦДА	2000	$\sim 7 \cdot 10^{-3}$	10
Брезенхем (отрезок)	2000	$\sim 4 \cdot 10^{-3}$	10
Брезенхем (окружность)	500	$\sim 30 \cdot 10^{-3}$	28

Анализ. Как и в эталонной веб-версии, алгоритм Брезенхема для отрезка оказывается быстрее ЦДА и пошагового алгоритма примерно в 2—2,5 раза — за счёт отсутствия делений и вызовов `js_round` (`math.floor`). Алгоритм для окружности выполняется заметно дольше из-за финальной дедупликации точек через `set` (до 8 вставок на итерацию плюс создание нового списка), однако внутри основного цикла также использует только целочисленную арифметику. Абсолютные значения для Python на порядок-два больше, чем для JavaScript (V8 оптимизирует такие циклы до машинного кода), но *относительный* порядок сложности сохраняется, что подтверждает теоретические оценки.

6 Реализация дополнительных возможностей

6.1 Анимация

При нажатии кнопки *Анимация* выполняется пошаговое построение: точки выводятся с интервалом ≈ 80 мс, а текущая строка таблицы растровых точек подсвечивается розовым тегом `cur`. Реализация основана на методе `root.after(80 tick)`, который ставит отложенную задачу в очередь главного цикла Tkinter; это позволяет визуально наблюдать последовательность работы алгоритма, не блокируя интерфейс.

6.2 Масштабирование

Регулятор масштаба (10–80 пикселей на единицу) позволяет исследовать поведение растеризации при разном разрешении растра. При изменении масштаба вся сцена перерисовывается без повторного вычисления массива точек — метод `_on_scale` вызывает только `render(self.points)`.

6.3 Замер времени

Кнопка *Замер времени* в цикле вызывает каждый алгоритм заданное число раз и выводит моноширинный отчёт в информационную панель: имя алгоритма, число повторов, среднее и суммарное время. Это позволяет сравнить эффективность методов в реальном времени на конкретной машине и убедиться в сохранении относительного порядка скоростей.

7 Заключение

В ходе выполнения лабораторной работы были реализованы четыре базовых растровых алгоритма: пошаговый, ЦДА, Брезенхема для отрезка и Брезенхема для окружности. Все алгоритмы корректно строят растровые примитивы, что подтверждается совпадением результатов с ручными расчётами. Разработано настольное приложение на Python/Tkinter с удобным интерфейсом, поддержкой сетки, осей координат, подписей, масштабирования, анимации и замера времени.

Сравнение временных характеристик подтвердило теоретические ожидания: целочисленный алгоритм Брезенхема для отрезка работает примерно вдвое-втрое быстрее ЦДА и пошагового алгоритма. Пояснена схема привязки целочисленных координат к дискретной сетке через преобразование $s_x = W/2 + xs$, $s_y = H/2 - ys$. Дополнительно отмечено, что при одинаковых правилах округления (`js_round`) пошаговый алгоритм и ЦДА дают идентичный растр на всех практически значимых входах, а расхождения с Брезенхемом возникают лишь в граничных случаях, когда идеальная линия проходит точно через середину пиксельной ячейки.

Полученные навыки могут быть применены при разработке графических редакторов, систем компьютерной графики и визуализации данных.

A Ключевые фрагменты кода

A.1 Алгоритм Брезенхема для отрезка

```
def bresenham_line(x1, y1, x2, y2):
    pts = []
    x, y = x1, y1
    dx = abs(x2 - x1)
    dy = -abs(y2 - y1)
    sx = 1 if x1 < x2 else -1
    sy = 1 if y1 < y2 else -1
    err = dx + dy
    while True:
        pts.append((x, y))
        if x == x2 and y == y2:
            break
        e2 = 2 * err
        if e2 >= dy:
            err += dy
            x += sx
        if e2 <= dx:
            err += dx
            y += sy
    return pts
```

Listing 2: bresenham_line(x1, y1, x2, y2)

A.2 Алгоритм Брезенхема для окружности

```
def bresenham_circle(cx, cy, r):
    pts = []
    if r < 0:
        return pts
    x, y, d = 0, r, 3 - 2 * r
    def add8(px, py):
        pts.extend([
            (cx + px, cy + py), (cx + py, cy + px),
            (cx + py, cy - px), (cx + px, cy - py),
            (cx - px, cy - py), (cx - py, cy - px),
            (cx - py, cy + px), (cx - px, cy + py),
        ])
    while x <= y:
        add8(x, y)
        if d < 0:
            d += 4 * x + 6
        else:
            d += 4 * (x - y) + 10
            y -= 1
        x += 1
    return unique_points(pts)
```

Listing 3: bresenham_circle(cx, cy, r)

A.3 Привязка координат к дискретной сетке

```
def world_to_screen(self, x, y):
    sx = W / 2 + x * self.scale
    sy = H / 2 - y * self.scale
    return sx, sy
```

Listing 4: world_to_screen(x, y)

A.4 Округление, совместимое с эталоном

```
def js_round(v):
    """Округление половиныотнуля  , как Math.round в JS."""
    return int(math.floor(v + 0.5))
```

Listing 5: js_round(v) — аналог Math.round

A.5 Выводы

В ходе работы было рассмотрено теоретическое обоснование основных алгоритмов растрового отображения отрезков на пиксельной сетке — пошаговый, DDA, Брезенхем.

Было реализовано демонстрационное приложение на языке программирования Python (среда исполнения — интерпретатор Python 3.13, графическая библиотека Tkinter из стандартной поставки).

Было установлено несовершенство алгоритмов растрового приближения как алгоритмов отображения сложных примитивов (кривые второго рода и старших родов) ввиду образования неестественно близких к прямым схем расположения пикселей в гладких строго выпуклых или вогнутых участках кривых. Было проведено сравнительное описание методов, найдены примеры отрезков с различным отображением в пиксельную сетку по разным алгоритмам (случай $(0, 0) \to (2, -1)$).

Был усвоен теоретический материал и найдено практическое применение показанных алгоритмов — а значит, поставленные цели лабораторной работы были выполнены.

В результате алгоритм Брезенхема продемонстрировал уверенное и стабильное ускорение относительно остальных двух алгоритмов в результате меньшего числа элементарных операций, что согласуется с теоретической оценкой сложности и подтверждается замером времени.